

Constructions JavaScript

Aperçu

Les cours de programmation préalables vous ont permis d'acquérir une compréhension fondamentale des structures de programmation courantes, notamment les opérateurs, les expressions, les structures de décision, les boucles et les fonctions. Cette activité porte sur quelques structures de contrôle essentielles.

Résultats d'apprentissage associés au cours

Démontrer une maîtrise de la syntaxe du langage JavaScript.

Préparer

Opérateurs
Expressions
Décisions
Boucles
Instructions d'activité

Opérateurs

En programmation, les opérateurs sont des symboles utilisés pour effectuer des opérations sur des opérandes (variables et valeurs) et pour traiter des expressions. De nombreux opérateurs réalisent des opérations mathématiques telles que l'addition, la soustraction, la multiplication et la division. D'autres gèrent l'affectation, la comparaison et les opérations logiques.

Opérateurs d'affectation

Les opérateurs d'affectation servent à attribuer des valeurs aux variables.

Opérateur	Description	Exemple	Retours
<code>==</code>	Égal à	<code>x == y</code>	vrai si x est égal à y
<code>===</code>	Strictement égal à	<code>x === y</code>	Vrai si x est égal à y et qu'ils sont du même type de données
<code>!=</code>	Non égal à	<code>x != y</code>	vrai si x n'est pas égal à y
<code>!==</code>	Strictement différent de	<code>x !== y</code>	Vrai si x n'est pas égal à y ou s'ils sont de types différents
<code><</code>	Moins que	<code>x < y</code>	vrai si x est inférieur à y
<code><=</code>	Inférieur ou égal à	<code>x <= y</code>	vrai si x est inférieur ou égal à y

Exemple:

```
let x = 5;
let y = 10;
x += y; // x is now 15 and y is still 10
```

Vous pouvez écrire du code JavaScript directement dans la console du navigateur pour le tester. Ouvrez les outils de développement, accédez à l'onglet **Console** , saisissez votre code et appuyez sur Entrée pour l'exécuter.

Démonstration vidéo : [20 secondes]

Points clés :

- L' `=` opérateur *attribue* une valeur ; il ne vérifie pas l'égalité.
- Le nom de la variable se place à **gauche** .
- La valeur (valeur littérale, variable ou expression) à affecter se place à **droite** .
- Après son affectation, la variable stocke la valeur et peut être utilisée dans tout votre programme.

Opérateurs arithmétiques

Les opérateurs arithmétiques servent à effectuer des opérations arithmétiques.

Opérateur	Description	Exemple	Retours
<code>&&</code>	ET logique	<code>x && y</code>	vrai si x et y sont tous deux vrais
<code> </code>	OU logique	<code>x y</code>	vrai si x ou y est vrai
<code>!</code>	Négation logique	<code>!x</code>	vrai si x est faux ; faux si x est vrai

Exemple:

```
let a = 7;
let b = 3;
let c = a + b; // c is now 10
let d = a * 100; // d is now 700
let e = a % b; // e is now 1
```

Points clés :

- Les opérateurs arithmétiques sont utilisés pour effectuer des calculs mathématiques.
- Ils peuvent être utilisés avec des valeurs littérales et des variables.
- La priorité des opérateurs détermine l'ordre des opérations dans les expressions complexes.

Opérateurs de comparaison

Les opérateurs de comparaison servent à comparer deux valeurs et à renvoyer une valeur booléenne (vrai/faux) en fonction de la comparaison.

Opérateur	Description	Exemple	Retours
<code>==</code>	Égal à	<code>x == y</code>	vrai si x est égal à y
<code>===</code>	Strictement égal à	<code>x === y</code>	

Vrai si x est égal à y et qu'ils sont du même type de données

!=

Non égal à

x != y

vrai si x n'est pas égal à y

!==

Strictelement différent de

x !== y

Vrai si x n'est pas égal à y ou s'ils sont de types différents

<

Moins que

x < y

vrai si x est inférieur à y

<=

Inférieur ou égal à

x <= y

vrai si x est inférieur ou égal à y

Exemple:

```
let x = 10;  
let y = 5;  
let z = x + y;  
console.log(z == 15); // true  
console.log(z === "15"); // false  
console.log(x < y); // false
```

Points clés :

- Les opérateurs de comparaison renvoient des valeurs booléennes (vrai/faux).
- Elles sont couramment utilisées dans les instructions conditionnelles et les boucles pour contrôler le déroulement du programme.
- L'égalité stricte (===) vérifie à la fois la valeur et le type, tandis que l'égalité souple (==) vérifie uniquement la valeur.

Opérateurs logiques

Les opérateurs logiques sont utilisés pour combiner plusieurs expressions booléennes et renvoyer une valeur booléenne basée sur l'opération logique composée.

Opérateur
Description
Exemple
Retours

&&

ET logique

x && y

vrai si x et y sont tous deux vrais

||

OU logique

x || y

vrai si x ou y est vrai

!

Négation logique

!x

vrai si x est faux ; faux si x est vrai

Exemple:

```
let a = true;
let b = false;
let c = a && b;
console.log(c); // false
let d = a || b;
console.log(d); // true
let e = !a;
console.log(e); // false
```

Points clés :

- Les opérateurs logiques sont utilisés pour combiner ou inverser des expressions booléennes.
- Ils sont essentiels pour contrôler le flux d'exécution du programme dans les instructions conditionnelles et les boucles.
- Évaluation en court-circuit avec **&&** et **||** arrête l'évaluation dès que le résultat est connu

Retour en haut de page

Une expression est une combinaison de valeurs, de variables, d'opérateurs et de fonctions dont l'évaluation produit une seule valeur. Les expressions servent à effectuer des calculs, à prendre des décisions et à manipuler des données en programmation.

Types d'expressions

- **Expressions arithmétiques** : Ces expressions font appel à des opérateurs arithmétiques (tels que +, -, *, /) pour effectuer des calculs mathématiques.
`let result = (5 + 3) * 2;`
- **Expressions de chaînes de caractères** : Ces expressions impliquent la concaténation ou la manipulation de chaînes de caractères à l'aide de l'opérateur + ou de méthodes de chaînes.
`let greeting = "Hello, " + "world!";`
- **Expressions logiques** : Ces expressions utilisent des opérateurs logiques (tels que &&, ||, !) pour évaluer des valeurs booléennes.
`let isAdult = age >= 18 && hasID;`

[Retour en haut de page](#)

Décisions

Les structures conditionnelles servent à prendre des décisions en programmation. Elles permettent au programme d'exécuter différents blocs de code selon qu'une condition spécifiée est vraie ou fausse. Les structures conditionnelles les plus courantes en JavaScript sont l'**if** instruction ``if``, l'**else** instruction ``else`` et l'**else if** instruction ``if``.

- **if instruction** : Exécute un bloc de code si une condition spécifiée est vraie.

```
if (condition) {  
    // Code to execute if the condition is true  
}
```

- La condition est évaluée à une valeur **booléenne**, c'est-à-dire vraie ou fausse.
Si la condition est vraie, le bloc de code à l'intérieur de l'instruction `if`

est exécuté. Si elle est fausse, le bloc de code est ignoré.

Exemple:

```
let age = 20;
if (age >= 18) {
  console.log("Price: Adult — you pay full price.");
}
```

- **else instruction** : Fournit un bloc de code alternatif à exécuter si la condition de l'instruction if est fausse.

```
if (condition) {
  // Code to execute if the condition is true
} else {
  // Code to execute if the condition is false
}
```

- **Exemple:**

```
let age = 16;
if (age >= 18) {
  console.log("Price: Adult — you pay full price.");
} else {
  console.log("Price: Child — you get a discount.");
}
```

- **else if Déclaration** : Cette structure permet de vérifier plusieurs conditions successivement.

```
if (condition1) {
  // Code to execute if condition1 is true
} else if (condition2) {
  // Code to execute if condition2 is true
} else {
  // Code to execute if none of the conditions are true
}
```

- **Exemple:**

```
let age = 65;
if (age < 13) {
  console.log("Price: Child — you get a discount.");
}
```

```

} else if (age <= 64) {
  console.log("Price: Adult — you pay full price.");
} else {
  console.log("Price: Senior — you get a discount.");

```

- `}]`
- **Instructions Switch** : Permettent d'exécuter des blocs de code sélectifs en fonction de la valeur d'une expression.

```

switch (expression) {
  case value1:
    // Code to execute if expression is equal to value1
    break;
  case value2:
    // Code to execute if expression is equal to value2
    break;
  // ... more cases ...
  default:
    // Code to execute if none of the cases match
}

```

- **Exemple:**

```

let day = 3;
switch (day) {
  case 1:
    console.log("Monday");
    break;
  case 2:
    console.log("Tuesday");
    break;
  case 3:
    console.log("Wednesday");
    break;
  default:
    console.log("Another day");
}

```

[Retour en haut de page](#)

Les boucles permettent de répéter un bloc de code plusieurs fois jusqu'à ce qu'une condition spécifiée soit remplie. Les structures de boucle les plus courantes en JavaScript sont la **for** boucle `for`, la **while** boucle `while` et la **forEach** boucle `forEach`.

- **for** Boucle : Répète un bloc de code un nombre de fois spécifié.

```
for (let i = 0; i < 10; i++) {  
  // Code to execute in each iteration
```

- }
- **Exemple:**

```
for (let i = 0; i < 5; i++) {  
  console.log("Iteration number: " + i);
```

- }
- **while** Boucle : Répète un bloc de code tant qu'une condition spécifiée est vraie.

```
while (condition) {  
  // Code to execute while the condition is true
```

- }
- **Exemple:**

```
let count = 0;  
while (count < 5) {  
  console.log("Count is: " + count);  
  count++;
```

- }
- **forEach** Boucle : Utilisée avec **les tableaux** ; elle parcourt chaque élément du tableau.

```
array.forEach(function(element) {  
  // Code to execute for each element
```

- });
- **Exemple:**

```
let fruits = ["apple", "banana", "cherry"];  
fruits.forEach(function(fruit) {  
  console.log("Fruit: " + fruit);
```

- `});`

[Retour en haut de page](#)

Instructions d'activité

1. Compte tenu des déclarations de variables suivantes :

```
const DAYS = 6;  
const LIMIT = 30;
```

2. `let studentReport = [11, 42, 33, 64, 29, 37, 44];`
3. Saisissez les extraits de code suivants dans la console de votre navigateur :
 - Écrivez une **for**boucle qui parcourra le **studentReport** tableau et affichera dans la console la valeur actuelle du tableau si elle est inférieure à 30.
 - Répétez le fragment de programme précédent en utilisant une **while**boucle.
 - Répétez le fragment de programme précédent en utilisant une **forEach**boucle.
 - Répétez le fragment de programme précédent en utilisant une **for...in**boucle.
 - Utilisez n'importe quel type de boucle pour générer dynamiquement les noms des jours (lundi, mardi, mercredi, etc.) pour les prochains **DAYS**jours à partir d'aujourd'hui.